# Small autoclave thermometry analysis report

By Helena Jackson

26th March 2026

# 1 Results for Thermometry Analysis

## Task 1: Sensor Noise

Table 1: Detrended Noise by Phase and Setpoint

| Target (°C) | Heating (°C) | Hold (°C) | Cooling (°C) |
|---|---|---|---|
| 127 | 0.377 | 0.269 | 0.380 |
| 135 | 2.486 | 1.032 | 1.891 |

**Explanation:** Sensor noise is higher at 135°C than at 127°C across all phases of the cycle. The hold phase (defined as within ±2°C of the target) consistently shows the lowest noise, while heating exhibits the largest fluctuations. This shows that noise depends on whether the system is in a dynamic or steady-state region. The increased noise at higher temperature suggests greater fluctuations at elevated operating conditions.

## Task 2: Time to Reach Process Temperature

Table 2: Time to Reach Setpoint

| Target (°C) | Mean (hr) | Std (hr) | $n$ |
|---|---|---|---|
| 127 | 3.984 | 2.079 | 7 |
| 135 | 5.199 | 0.726 | 9 |

**Explanation:** On average, the autoclave reaches 127°C in approximately 4.0 hours and 135°C in approximately 5.2 hours. The higher-temperature program takes about 1.2 additional hours. The larger standard deviation for 127°C runs indicates greater variability between cycles at the lower setpoint. Overall, the data show that increasing the target temperature leads to a consistently longer heat-up time.

*Here, $n$ represents the number of valid autoclave cycles included in each grouped calculation after applying duration and data-filtering criteria.*

## Task 2: Cooling Time to ~35°C

Table 3: Cooling Time from Peak to 35°C

| Target (°C) | Mean (hr) | Std (hr) | $n$ |
|---|---|---|---|
| 127 | 7.660 | 3.802 | 6 |
| 135 | 12.077 | 1.479 | 4 |

**Explanation:** Cooling from peak temperature to approximately 35°C takes substantially longer for the 135°C program than for the 127°C program. The average cooling time increases by more than four hours at the higher setpoint. This is consistent with greater thermal energy at higher temperatures. Some

runs did not reach 35°C within the recorded log duration, which reduced the number of valid cooling measurements.

### Task 3: Accuracy Relative to Intended Setpoint

Table 4: Hold Region Temperature Statistics

| Target (°C) | Mean Hold (°C) | Hold Error (°C) | Std (°C) | $n$ |
|---|---|---|---|---|
| 127 | 127.682 | +0.682 | 0.203 | 7 |
| 135 | 134.976 | -0.024 | 0.232 | 9 |

**Explanation:** Both process programs maintain temperatures very close to their intended setpoints during the hold region. The 127°C runs show a small positive bias of approximately +0.68°C, meaning the system slightly overshoots its target. In contrast, the 135°C runs are nearly centered on the intended temperature, with an average error close to zero. The relatively small standard deviations indicate stable temperature control once the system reaches steady-state conditions.

## 1.1 Thermometry Analysis Code

The full thermometry analysis code is included in Appendix A. The main functions used to clean the data, identify the target temperature, and calculate the cycle metrics are shown below.

```
def load_clean(path):
    df = pd.read_csv(path, comment="#", header=None, names=["ctime", "temp"])
    df = df[(df["temp"] >= T_MIN) & (df["temp"] <= T_MAX)].copy()
    df = df.sort_values("ctime").reset_index(drop=True)

    if df.empty:
        return None

    df["t_hr"] = (df["ctime"] - df["ctime"].iloc[0]) / 3600.0
    return df

def infer_target(df):
    t95 = df["temp"].quantile(0.95)
    return 135 if t95 > 132 else 127

def analyze_one(path):
    df = load_clean(path)
    if df is None or duration_hr(df) < MIN_HOURS:
        return None

    df = add_slope(df)
    target = infer_target(df)

    t_reach = time_to_reach(df, target)
    t_cool35 = time_to_cool35_after_peak(df)

```

```python
    hold = df[(df["temp"] >= target - 2) & (df["temp"] <= target + 2)]
    hold_mean, hold_std, hold_p2p = segment_stats(hold["temp"])
    hold_err = hold_mean - target if np.isfinite(hold_mean) else np.nan

    ramp = df[df["dT"].abs() > 0.05]
    ramp_mean, ramp_std, ramp_p2p = segment_stats(ramp["temp"])

    i_peak = int(df["temp"].idxmax())
    cool_seg = df.loc[i_peak:]
    cool_mean, cool_std, cool_p2p = segment_stats(cool_seg["temp"])

    return {
        "target_C": target,
        "time_to_reach_hr": t_reach,
        "time_to_cool35_after_peak_hr": t_cool35,
        "hold_mean_C": hold_mean,
        "hold_std_C": hold_std,
        "hold_error_C": hold_err,
        "ramp_std_C": ramp_std,
        "cooling_std_C": cool_std,
    }

def rolling_noise(series, window=61):
    trend = series.rolling(window, center=True,
                           min_periods=window//2).median()
    resid = series - trend
    return resid.std(ddof=1)
```

# 2 Results for Cycle Alignment

## 2.1 Overview

The primary objective of this work was to implement a time-aligned cycle representation, converting raw autoclave temperature logs into a consistent format where each row corresponds to a complete cycle from start to end. This structure enables direct comparison between cycles and between the 127 °C and 135 °C programs.

## 2.2 Code Implementation and Cycle Extraction

All `.log` files were first cleaned by filtering temperatures to a realistic range (20–150 °C) and removing runs shorter than 10 hours. Time was then normalized so that each cycle begins at $t = 0$, allowing all runs to be aligned on the same time axis.

The target temperature (127 °C vs. 135 °C) was inferred from the data, and each cycle was segmented into key regions based on physical behavior:

- **Heating (ramp):** identified using a temperature slope threshold ($|dT/dt| > 0.05$ °C/s), capturing dynamic temperature changes.
- **Hold:** defined as temperatures within ±2 °C of the target, representing steady-state operation.
- **Cooling:** defined as all data following the peak temperature.

From these segments, key metrics were computed, including time to reach setpoint, cooling time to ∼35 °C, and temperature stability during the hold phase. Noise was further estimated using detrended residuals from a rolling median, which isolates short-term fluctuations.

Finally, all runs were compiled into a single dataset where each row represents one full cycle. Summary statistics were then calculated by target temperature to enable direct comparison between programs.

## 2.3 Cycle Alignment Code

The full cycle-alignment code is included in Appendix A. The key functions used to align each cycle to a common starting point and build the final 2D cycle matrix/array are shown below.

```python
def load_clean_log(path):
    df = pd.read_csv(path, comment="#", header=None, names=["ctime", "temp"])
    df["ctime"] = pd.to_numeric(df["ctime"], errors="coerce")
    df["temp"] = pd.to_numeric(df["temp"], errors="coerce")
    df = df.dropna(subset=["ctime", "temp"])
    df = df[(df["temp"] >= T_MIN) & (df["temp"] <= T_MAX)].copy()

    if df.empty:
        return None

    df = df.sort_values("ctime").reset_index(drop=True)
    t0 = df["ctime"].iloc[0]
```

```python
    df["t_sec"] = df["ctime"] - t0
    df["t_hr"] = df["t_sec"] / 3600.0
    return df

def arp_extract_cycle(path, temp_start=40):
    df = load_clean_log(path)
    if df is None or duration_hr(df) < MIN_HOURS:
        return None

    target = infer_target(df)
    idx_start = (df["temp"] > temp_start).idxmax()

    if idx_start == 0:
        return None

    df["t_sec"] = df["t_sec"] - df["t_sec"][idx_start]
    df = df[df["t_sec"] <= CYC_TIME_SEC].copy()

    if df.empty:
        return None

    max_t = int(df["t_sec"].max())
    common_t_sec = np.arange(0, max_t + 1, DT_SECONDS)
    common_temp = np.interp(common_t_sec, df["t_sec"].values,
        df["temp"].values)

    return {
        "target_C": target,
        "time_sec": common_t_sec,
        "time_hr": common_t_sec / 3600.0,
        "temp": common_temp
    }

def build_cycle_matrix(cycle_list):
    min_len = min(len(c["temp"]) for c in cycle_list)
    cycle_matrix = np.vstack([c["temp"][:min_len] for c in cycle_list])
    time_hr = cycle_list[0]["time_hr"][:min_len]
    return cycle_matrix, time_hr
```

## 2.4 Cycle Comparison Plots

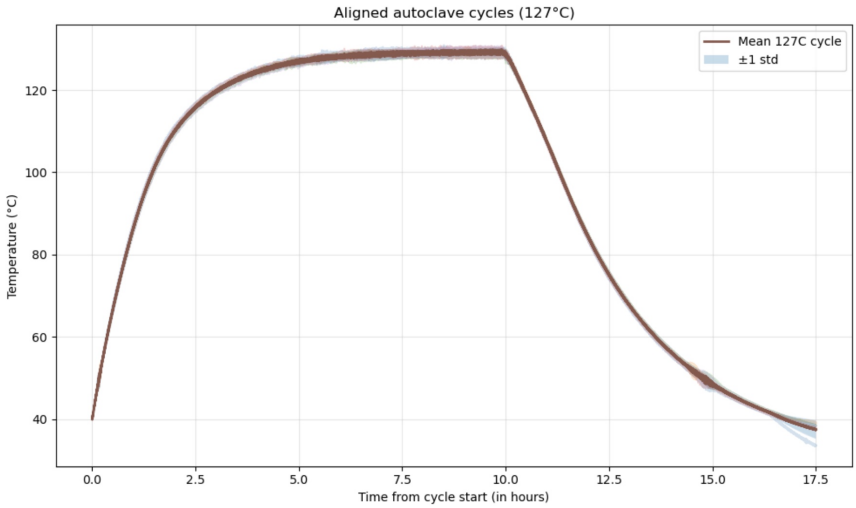


Figure 1: Aligned autoclave cycles for the 127°C program. Individual cycles are shown along with the mean profile and ±1 standard deviation band.

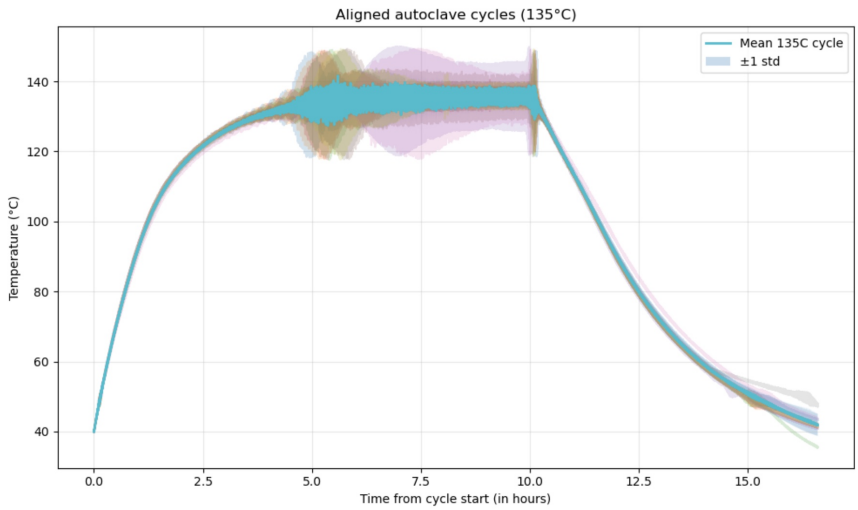


Figure 2: Aligned autoclave cycles for the 135°C program. Individual cycles are shown along with the mean profile and ±1 standard deviation band.

The aligned cycle plots show the heating, hold, and cooling behavior for both programs. The 127°C cycles are tightly grouped, indicating consistent behavior across runs. In contrast, the 135°C cycles show more spread during the hold region, along with visibly higher noise. This suggests that the system is less stable at the higher setpoint, particularly during steady-state.

# Appendix A Full Code Files

The full code files.

Jupyter Notebook:

- `autoclave_work(1).ipynb`
- `Plot_data_CMB 2026 revised.ipynb`

Google Colab:

- Notebook 1: autoclave work
- Notebook 2: cycle alignment code